

LCTA 技術說明

柴比契夫取樣點法：

針對 3.2.1 三種離散化技術比較之結果，本論文擬採用柴比契夫取樣點法執行專案網路離散化相關之運算，柴比契夫取樣點法與其它取樣技術最大的不同處，是其樣本點的取點位置很特殊，首先在時間軸上訂出欲離散化之機率分佈函數的資料範圍[a, b] (參考圖 3.3)，a, b 兩點的決定可由隨機變數之機率分佈函數取其 0.05 及 0.95 累積機率之時間軸位置，並以該範圍直線距離為直徑畫一圓，其取樣點是均勻等距分佈在圓周上，再投影於時間軸上，該投影的位置就是柴比契夫取樣點法取樣點位置(若不包括兩端點，共取 5 個樣本點)。

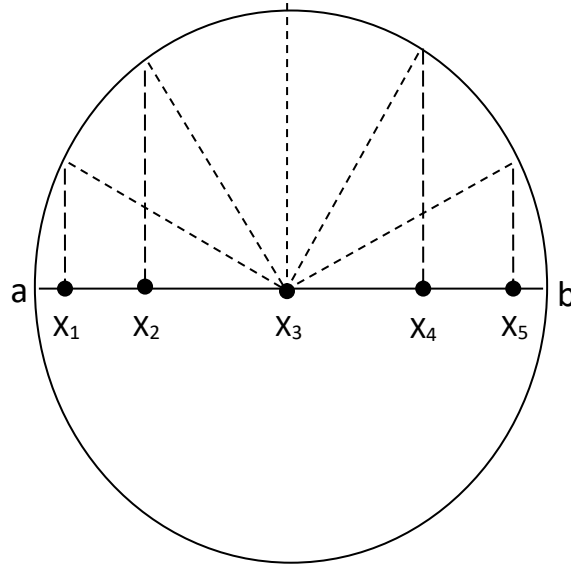


圖 3.3 柴比契夫取樣點法之取樣點圖示

柴比契夫取樣點法主要的取樣步驟表示如下：

1. [a,b]為作業時間資料範圍，利用下列之公式尋找 Q 個機率分佈樣本點：

$$x_i = a + \frac{(b-a)}{2} \left[1 - \cos\left(\frac{\pi(2i-1)}{2Q}\right) \right], \quad i=1, \dots, Q \quad (3.7)$$

2. 依據作業時間的機率分佈型態及其所給定之參數資料(機率分佈之 moments 值 $m_k, k=1, \dots, 4$)，利用下式求解相對於 x_i 離散資料之機率值 p_i 。

$$\begin{aligned}\sum_{i=1}^5 P_i &= 1 \\ \sum_{i=1}^5 x_i^k P_i &= m_k, \quad j=1, \dots, 4\end{aligned}\tag{3.8}$$

根據上式，一般作業時間機率分佈通常提供 4 個動差(Moment) m_k ，故柴比契夫取樣點法所取樣的點數為 $Q=5$ 個，當離散機率分佈能夠符合原機率分配之前 4 個動差值的前提下，該離散機率分佈即與原式相去不遠。

範例 3.1:

針對平均值 $\text{mean}=2$ 之指數分佈函數，資料範圍為 $[0, 20]$ ，使用柴比契夫取樣點法取 5 個樣本點，執行該函數之離散化。

執行步驟:

1. 先計算出該函數之前四個動差值，分別為 $[2.0, 8.0, 47.5, 372.77]$ 。
2. 依據資料範圍 $[0, 20]$ 及式(3.7)，計算出 5 個時間樣本點值為 $[0.49, 4.12, 10.0, 15.88, 19.51]$ 。
3. 再依據式(3.8)之聯立方程式計算出相對應之 5 個取樣點值，求得離散化之樣本點為

$$\begin{bmatrix} 0.49 & 4.12 & 10.0 & 15.88 & 19.51 \\ 0.56 & 0.41 & 0 & 0.03 & 0 \end{bmatrix}$$

離散化 max 及 convolution 運算原理：

存在兩專案時間變數 X, Y 之機率分配函數，其取樣點數分別為 Q 及 R ，離散化後可以表示為：

$$X = \begin{bmatrix} x_1, x_2, \dots, x_Q \\ p_1^x, p_2^x, \dots, p_Q^x \end{bmatrix}, \quad Y = \begin{bmatrix} y_1, y_2, \dots, y_R \\ p_1^y, p_2^y, \dots, p_R^y \end{bmatrix}\tag{3.9}$$

$$x_1 < x_2 < \dots < x_Q, \quad p_i^x > 0, \quad i=1, \dots, Q,$$

$$y_1 < y_2 < \dots < y_R, \quad p_j^y > 0, \quad j=1, \dots, R$$

X, Y 兩變數均表示為矩陣型態，第一列表示變數的函數值，第二列則表示相對應之機率值，該兩變數之離散化 max 運算結果 $Z=\max(X, Y)$ 可以表示如下：

$$Z = \begin{bmatrix} z_1, z_2, \dots, z_s \\ p_1^z, p_2^z, \dots, p_s^z \end{bmatrix}\tag{3.10}$$

$$z_k = z_{ij} = \max(x_i, y_j), \quad p_k^z = \sum_{(i,j) \in T} p_i^x p_j^y, \quad T = \{(i, j) \mid z_{ij} \text{ has same value}\}$$

max 相關的運算方法以下列程序來做說明：

1. 先取 X, Y 中每一樣本函數值來做 $\max$ 運算，可得 $z_{ij} = \max(x_i, y_j)$ 。所得之 Z 變數樣本函數值 $z_k = z_{ij}$ 相對應之機率值為 $p_{ij}^z = p_i^x p_j^y$ ，其中 $i=1, \dots, Q, j=1, \dots, R$ ，且 Z 變數最多可有 $W = \max(Q, R)$ 樣本單元。

2. 將求出之 Z 變數每一樣本函數值 (z_{ij}, p_{ij}^z) ，依 z_{ij} 從小至大做排序。

3. 將 z_{ij} 值相同的樣本函數值合併為一，並表示為 z_k ，且其中相對應之 $p_i^x p_j^y$ 要加總

$$p_k^z = \sum_{(i,j) \in T} p_i^x p_j^y。$$

從上述可知， $\max$ 運算第一步驟需執行 $Q \times R$ 次 $\max(x_i, y_j)$ 運算，第二步驟則須執行 $W \ln W$ 次排序工作，第三步驟亦同樣需要 W 次合併比較。 convolution 運算亦同 $\max$ 運算方式執行，其中僅將第一步驟中之 $z_{ij} = \max(x_i, y_j)$ 改為 $z_{ij} = x_i + y_j$ 即可，且 Z 變數最多可有 $Q \times R$ 樣本單元。運算的複雜度亦同 $\max$ 運算。

離散化重新取樣技術：

就實務上而言，執行 X, Y 變數離散化 convolution 及 $\max$ 運算，所得之結果 $Z = \{z_{ij}, i=1, \dots, Q, j=1, \dots, R\}$ ，若執行相同的 z_{ij} 樣本單元合併可以減少樣本單元數。然而 z_{ij} 樣本單元數值通常為帶小數之實數，因此 z_{ij} 具同樣本單元之機率並不大。在此情況下， X, Y 變數間執行 convolution 及 $\max$ 運算所得到的 z_{ij} 樣本單元數目一般情況為 $Q \cdot R$ ，若運算持續執行若干次，其所得結果之樣本單元數目會成長得非常快速。參考 3.2.3 節 convolution 及 $\max$ 運算所需花費的執行步驟，樣本單元數過大仍會造成離散化運算之負擔，這會嚴重影響專案網路整體估算之效率。因此必須要降低該樣本單元數目，同時要維持變數相關數值(平均值、變異數)之精確度。本論文擬以上述所求之 Z 變數為例，將其所得到的 z_{ij} 樣本單元(假設共有 $Q \cdot R$ 個)重新取樣，並將重新取樣的樣本數定為 S ，即將(3.10)式轉換成：

$$Z' = \begin{bmatrix} z'_1, z'_2, \dots, z'_S \\ p'_1, p'_2, \dots, p'_S \end{bmatrix}, S < QR \quad (3.11)$$

相關的執行步驟如下：

1. 將 Z 變數數值分割成 S 個區間，區間的間隔為 $\Delta = (z_w - z_1)/S$ 。每一區間含原 Z 變數之數值單元為

$$2. D_k = \{z_d | z_d \in (z_1 + (k-1)\Delta, z_1 + k\Delta], k=1, \dots, S\} \quad (3.12)$$

3. 每一區間以新的數值單元 (z'_k, p'_k) ， $k=1, \dots, S$ 來表示，其中

$$4. z'_k = \sum_{D_k} z_d \cdot p_d^Z / \sum_{D_k} p_d^Z, \quad p_k^{Z'} = \sum_{D_k} p_d^Z \quad (3.13)$$

5. 若區間沒有存在樣本點，則該區間所產生之新的單元數值，可表示為

$$(z'_k, p_k^{Z'}) = (z_1 + (k - 0.5)\Delta, 0) \quad (3.14)$$

當以離散化技術執行 convolution 及 max 運算，其結果之單元數值數目若超過 S 個，則執行重新取樣技術(Discrete Resample Technique, DRT)，如此就能夠有效的控制單元數值數量過大的問題。

“路徑相依性”造成專案網路估算之偏差說明：

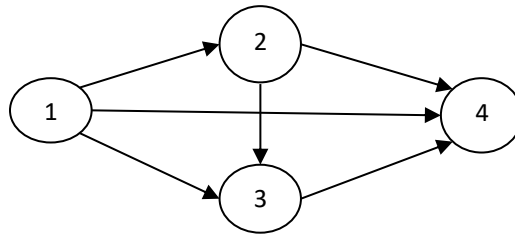


圖 1. 範例網路

定義 $Output_i$ 為 node(i) 的輸出時間，計算網路的完成時間，一般程序如下：

1. $Output_2 = Output_1$
2. $Output_3 = \max[Output_2, Output_1]$
3. $Output_4 = \max[Output_3, Output_2, Output_1]$

估算過程中需要進行 2 次隨機變數 max 運算

專案網路路徑的相依性是源於路徑間使用共同之節點，以圖 1 所例，路徑(1-2-4)與(1-2-3-4)有重複的節點{1,2}，路徑(1-3-4)與(1-2-3-4)之重複的節點為{1,3}。

這些重複的節點會讓隨機網路中時間估算產生偏差，主要原因乃源於估算過程中 max 運算帶來的結果，讓專案網路完成時間高估於實際值。

因為隨機變數間的 max 運算與一般數值的 max 運算(直接選取最大值)不相同，其特色說明如下：

1. 會相當受到變數內“變異數(variance)”的影響，且變異數愈大，影響愈大！
2. 若兩個隨機變數均為常態分佈，兩變數間的 max 運算結果不一定為常態分佈！
3. 網路中若兩路徑有重疊現象發生，該兩路徑進行 max 運算，若採用上述一般運算方式處理，便會造成本文所謂的“相依路徑偏差”，且估算出來的結果通常比實際值要大！

4. 實務界通常改用 PERT 來進行估算，但若遇到網路相依狀態出現，同樣也會出現明顯的估算誤差(比實際值要小)
5. 本文採用蒙地卡羅模擬法，以 20000 次取樣進行模擬計算隨機網路時間，其結果非常接近於實際值，也以此標準來做比較

本文另以圖 2 為範例說明：

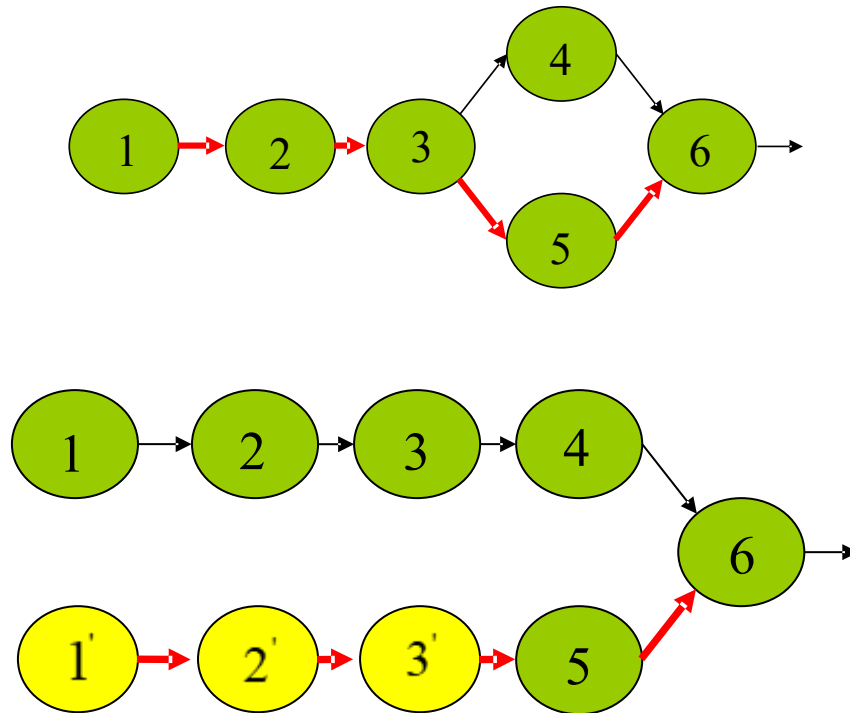


圖 2. 網路路徑相依說明圖例

一般程序計算方式：

1. $Output_3$ = 是節點 1~3 累加的隨機變數時間(累積採用 convolution 運算)
2. $Output_4 = Output_3 \oplus Node_Time(4)$
3. $Output_5 = Output_3 \oplus Node_Time(5)$
4. $Output_6 = \max\{Output_4, Output_5\} \oplus Node_Time(6)$

上述的計算方式會出現“路徑相依性偏差”，解釋如下：

路徑(1-2-3-4-5-7)及(1-2-3-4-6-7)為兩獨立之路徑，即該兩路徑之完成時間之變異數為該路徑上所有作業時間變異數之加總，然而子路徑(1-2-3-4)為該兩路徑共同持有，該子路徑時間之變異數，針對節點 6 執行 max 運算僅能“影響一次”(這是很重要的概念，許多演算法都忽略這個重要的因素)！然而上述的計算方式，將該重複路徑的“變異數”在 max 運算過程“採用了兩次”，max 運算多了一次子路徑(1-2-3-4)變異數之影響，產生“路徑相依性偏差”。

路徑相依因素納入 LCTA 演算法：

解決路徑相依偏差也很簡單，只要將上述子路徑(1-2-3-4)變異數的影響移出，也就是將重複子路徑內的"variance"成分去除即可！

演算流程概述：

1. 首先將專案網路結構各完成路徑間"重複路徑"找出
2. 專案網路中執行 max 運算的節點，檢查輸入路徑中是否存在"重複路徑"
3. 若存在"重複路徑"， max 運算只允許其中一條路徑的隨機時間變數帶有"variance"，去除其餘重複路徑的"variance"！
4. 去除 variance 方式：

令 Z 為一隨機變數： $Z = [v1, v2, \dots : p1, p2, \dots]$

將 Z 轉變為： $[E(Z), 1]$ 隨機變數型態， $E(Z)$ 為 Z 變數的期望值

理解了上述的方式後，我們在重新進行圖 2 的計算：

1. $Output_3 =$ 是節點 1~3 累加的隨機變數時間(累積採用 convolution 運算)
2. $Output_4 = Output_3 \oplus Node_Time(4)$
3. $Output'_3 = [E(Output_3), 1]$ // 去除其中一條重複路徑的 variance
4. $Output_5 = Output'_3 \oplus Node_Time(5)$
5. $Output_6 = \max\{Output_4, Output_5\} \oplus Node_Time(6)$

LCTA 演算法已將上述的路徑相依因素納入考量，巧妙的使用"Shared_flag"旗標，安置於 Downward_Tracing(n)函式及 Upward_Tracing()函式中(請參考 LCTA 演算法流程)